Enunciado evaluación Formativa 1
Programación Orientada a Objetos

FORMA A

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1102	Programación Orientada a Objetos	EA1: Fundamentos de Programación Orientada a Objetos
Tiempo	**Modalidad de Trabajo**	**Indicadores de logro**
3 bloques	Individual	IL 1.1 al IL 1.6

Condiciones de la evaluación

Del entorno y la entrega

- La evaluación es de carácter individual. No está permitida ninguna forma de colaboración entre estudiantes.
- No se permite el acceso a internet ni el uso de teléfonos, relojes inteligentes ni ningún otro dispositivo externo.
- El programa debe compilar y ejecutarse correctamente. Una solución que no compile no permite evidenciar ningún indicador de logro.
- El proyecto debe entregarse comprimido en formato ZIP, con tu nombre completo como nombre del archivo, en la actividad habilitada en AVA antes de que finalice la sesión.
- No se aceptan entregas por ningún otro medio ni fuera del horario establecido. La ausencia de entrega en AVA genera automáticamente la nota mínima.

De tu solución

- El código debe organizarse en clases separadas, respetando las convenciones de nomenclatura de Java: PascalCase para nombres de clase, camelCase para métodos y atributos.
- Puedes crear métodos o clases auxiliares siempre que no contradigan los requerimientos del caso.
- No se admite el uso de herramientas de inteligencia artificial ni de recursos digitales externos.
- Cualquier indicio de colaboración no autorizada, uso de IA o plagio implica la anulación inmediata de la evaluación con nota mínima de 1.0.

Instrucciones de entrega

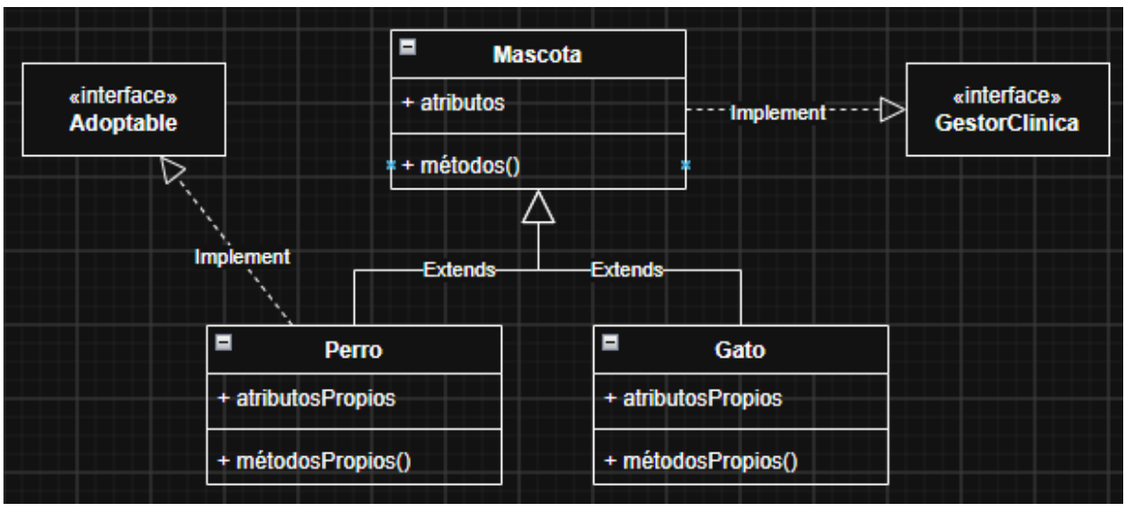
- Comprime el proyecto completo (todas las clases) en un único archivo ZIP.
- Nombra el archivo con tu nombre completo, sin espacios ni tildes (ejemplo: Juan_Perez_Lopez.zip).
- Sube el archivo a la actividad de entrega habilitada en AVA antes de que finalice la sesión asignada.
- No se aceptan entregas por correo electrónico, USB ni ningún otro medio.
- La ausencia de entrega en AVA genera automáticamente la nota mínima, independientemente del avance de la solución.

1. Contexto del caso

La Clínica Veterinaria PataVerde requiere un sistema para administrar los animales que atiende y registrar el costo de cada consulta. La clínica trabaja con distintos tipos de animales; aunque todos comparten información básica como nombre, edad y peso, el costo de consulta varía según el tipo de animal. Adicionalmente, ciertos animales son candidatos a procesos de adopción, lo que implica gestionar su disponibilidad de forma diferenciada.

El sistema debe modelar esta situación aplicando los principios de la Programación Orientada a Objetos, de manera que el comportamiento compartido y el comportamiento específico de cada tipo de animal queden correctamente organizados en la jerarquía de clases.

Diagrama de clases



2. Información necesaria

El sistema contempla las entidades que se muestran en el diagrama. Cada clase debe incluir constructor(es), métodos accesadores (getters), mutadores (setters) y el método **toString(): el cual debe incluir únicamente el nombre y la edad del animal**. Los constructores deben delegar la asignación de valores a los métodos setter, de modo que las validaciones definidas se apliquen desde la construcción del objeto.

- Todos los **animales** del sistema comparten nombre, edad y peso. Dado que el costo de consulta se determina de manera diferente para cada tipo de animal, el sistema debe contemplar un mecanismo que permita calcular ese costo de forma general sin conocer el tipo específico del objeto en cada momento. En esta clase padre, debe existir también como atributo la lista de animales que necesitan gestionar la clínica..
- Los **perros** agregan raza, estado de vacunación y un indicador de disponibilidad para adopción. Al participar en el proceso de adopción de la clínica, deben implementar el contrato definido por la interfaz **Adoptable**.
- Los **gatos** agrega un atributo que indica si el animal vive en exterior, condición que incide en el costo de la consulta.
- El **gestor de la clínica** es la interfaz responsable de definir los métodos que van a administrar la colección de animales registrados y exponer las operaciones sobre ella. En ella deben estar los métodos que permitan: agregar, modificar o quitar objetos a la colección, además de listar (mostrar todos sus objetos)

3. Métodos especializados

El sistema debe ser capaz de calcular el costo de consulta para cualquier animal registrado, independientemente del tipo. Cada tipo de animal determina ese costo según su propia lógica:

- **Perro:** el costo base es $15.000. Si el animal no se encuentra vacunado, ese costo se incrementa en un 30%.
- **Gato:** el costo base es $12.000. Si el animal vive en exterior, el costo se incrementa en un 15%.

La interfaz Adoptable define el contrato que deben cumplir los animales que participan en el programa de adopción de la clínica. Está compuesta por dos operaciones:

- La primera permite consultar en cualquier momento si el animal se encuentra disponible para ser adoptado.
- La segunda permite registrar que el animal está disponible para adopción.

Esta interfaz está diseñada de manera que, en el futuro, cualquier tipo de animal pueda incorporarse al programa sin necesidad de modificar la jerarquía de clases existente. Por ahora, solo los perros implementan este contrato.

4. Validaciones esperadas

Las validaciones deben aplicarse en los métodos setter de cada clase. Cuando una condición no se cumpla, debe lanzarse una excepción de tipo **IllegalArgumentException** con un mensaje descriptivo que explique el motivo del rechazo.

- **nombre:** no puede ser nulo ni vacío.
- **edad:** debe encontrarse en el rango entre 0 y 30 años.
- **peso:** debe ser un valor mayor que cero.

5. Operaciones del sistema

La clase que gestiona la colección de animales debe exponer las siguientes operaciones:

- Registrar un animal en la colección e informar por consola que fue incorporado correctamente.
- Buscar y retornar los animales cuyo nombre coincida con el criterio de búsqueda recibido.

6. Requerimientos de la clase principal

La clase principal debe demostrar el funcionamiento completo del sistema e incluir los siguientes elementos:

- Instanciar un animal de cada tipo disponible utilizando los datos de la tabla siguiente.
- Marcar a Max (Perro) como disponible para adopción una vez creado.
- Registrar todos los animales en el gestor del sistema.
- Invocar las operaciones del gestor y mostrar sus resultados: buscar por nombre y listar todas las mascotas registradas mediante el método toString() de cada objeto.

Datos para la clase principal:

Tipo	Nombre	Edad	Peso (kg)	Atributo específico
Perro	Max	3	15.5	Raza: Labrador | Vacunado: true | Adopción: Disponible
Perro	Rex	5	8.0	Raza: Beagle | Vacunado: false
Gato	Max	2	4.2	Exterior: false
Gato	Luna	4	5.1	Exterior: true

7. Resultado esperado

La ejecución del programa con los datos indicados debe producir una salida similar a la siguiente:

```
Max (Perro) registrado correctamente.
Rex (Perro) registrado correctamente.
Max (Gato) registrado correctamente.
Luna (Gato) registrado correctamente.

=== BÚSQUEDA POR NOMBRE: "Max" ===
Tipo: Perro | Nombre: Max | Edad: 3 años | Peso: 15.5 kg | Raza: Labrador | Vacunado: Sí | Adopción: Disponible para adopción | Costo consulta: $15000
---
Tipo: Gato | Nombre: Max | Edad: 2 años | Peso: 4.2 kg | Vive en: Interior | Costo consulta: $12000
---

=== LISTADO DE MASCOTAS ===
Nombre: Max | Edad: 3 años
Nombre: Rex | Edad: 5 años
Nombre: Max | Edad: 2 años
Nombre: Luna | Edad: 4 años

Process finished with exit code 0
```

Se aceptan pequeñas diferencias en los textos o en el formato de los números, siempre que se muestren todos los datos solicitados y que los resultados numéricos sean correctos.